

1. Methodology

1.1. Road Network Representation

The foundation of this study is a detailed, real-world road network graph representing the urban topology of Kathmandu and Lalitpur, Nepal. The graph was constructed from OpenStreetMap (OSM) data using the `osmnx` Python library, a tool designed for street network analysis. The resulting network is modeled as a directed graph using the `networkx` library, where nodes correspond to intersections or road endpoints, and edges represent the road segments connecting them. Each node is geographically referenced with latitude and longitude coordinates, and each edge is weighted by its physical length, providing the cost metric for pathfinding calculations. To ensure efficient application startup and eliminate redundant data processing, the fully constructed graph is serialized into a `.gpickle` file, allowing for near-instantaneous loading.

1.2. Core Pathfinding Algorithm

The fundamental pathfinding mechanism employed in this system is the Bidirectional A* algorithm. This approach enhances the efficiency of traditional A* search by simultaneously exploring paths from both the

source and destination nodes, converging at a meeting point in the middle. This bidirectional strategy significantly reduces the search space compared to a unidirectional search, particularly in large graphs like

urban road networks. The heuristic function guiding both forward and backward searches is the Haversine distance. This function calculates the great-circle distance between two geographic points (latitude,

longitude) on a sphere, providing an admissible (never overestimating the actual cost) and consistent estimate of the shortest possible path. The Haversine heuristic provides a pure distance estimate, while the

obstacle avoidance logic is integrated directly into the main search algorithm's decision-making process, as detailed in Section 1.4.

1.3. Real-time Obstacle Integration

■ A critical aspect of this real-time urban navigation system is its ability to dynamically integrate and respond to unexpected road obstacles. Obstacle data, representing temporary road closures, construction, or ■

other impediments, is managed and stored in a Supabase database. These obstacles are identified by specific `node_ids` within the road network graph. To ensure the pathfinding algorithm operates with the most ■

current information while minimizing database query latency and load, an in-memory obstacle cache is implemented. This cache is periodically refreshed (e.g., every 5 minutes, configurable via `OBSTACLE_CACHE_TTL`) ■

from the Supabase database. This mechanism allows the system to rapidly incorporate newly reported or resolved obstacles into subsequent pathfinding calculations, enabling adaptive rerouting.

1.4. Obstacle Avoidance Mechanism

This section details the primary enhancement to the Bidirectional A* algorithm: its obstacle-aware avoidance mechanism, which constitutes the core contribution of this research. The strategy employed is *hard

exclusion*, effectively assigning an infinite penalty to any path segment (node or edge) that is deemed impassable or unsafe due to the presence of obstacles. This mechanism ensures that the algorithm strictly

avoids areas affected by real-time impediments.

The implementation of this avoidance strategy is critically dependent on a highly optimized Spatial Index (further elaborated in Section 1.5), which provides rapid, near $O(1)$ lookups for obstacle proximity.

During the pathfinding process, before considering any potential neighbor node or edge for path extension, the algorithm performs three critical checks:

1. Direct Obstacle Check: It verifies if the candidate node itself is explicitly marked as an obstacle (`spatial_index.is_obstacle`).

2. Node Proximity Check: It determines if the candidate node falls within a configurable `obstacle_radius` (e.g., 0.1 km) of any known obstacle (`spatial_index.is_near_obstacle`).

3. Edge Proximity Check: It assesses if the road segment (edge) connecting the current node to the candidate neighbor passes within the specified `obstacle_radius` of any obstacle (`spatial_index.is_edge_near_obstacle`). This check utilizes geometric analysis for precise detection.

If any of these conditions are met, the algorithm immediately discards that neighbor and does not consider it for path extension. This strict avoidance ensures that generated routes are entirely clear of

identified obstacles and their immediate vicinity. The `obstacle_radius` parameter offers flexibility in defining the buffer zone around obstacles, allowing for adaptation to varying obstacle types and safety requirements.

1.5. Performance Optimizations

To ensure the real-time responsiveness required for dynamic urban navigation, several performance optimization techniques have been integrated into the system, complementing the enhanced pathfinding algorithm.

Firstly, a custom Spatial Index was developed to accelerate geographic queries and graph traversal operations. This index leverages a k-d tree (`scipy.spatial.cKDTree`) to replace linear searches with highly

efficient spatial lookups. It provides near $O(1)$ or $O(\log n)$ complexity for critical operations such as verifying node existence, retrieving node coordinates, accessing edge data, and, most importantly, executing

all obstacle-related checks (`is_obstacle`, `is_near_obstacle`, `is_edge_near_obstacle`). This significantly reduces the computational overhead during the Bidirectional A* algorithm's graph exploration.

■

Secondly, a Path Caching mechanism is implemented to avoid redundant computations for frequently requested paths. Once a path is successfully computed, it is stored in an in-memory cache. The cache key is ■

intelligently generated based on the source node, destination node, and the current configuration of active obstacles. This design ensures that if an identical pathfinding request is made with the same obstacle ■

setup, the previously computed result can be returned instantly, drastically reducing computation time and improving overall system throughput and responsiveness.

Updated

1.1. Road Network Representation

The efficacy of any network-based pathfinding algorithm is fundamentally dependent on the fidelity of its underlying graph representation. For this study, a high-fidelity, directed graph was constructed to model

the complex urban road network of Kathmandu and Lalitpur, Nepal. This region was selected as its dense and often irregular street topology provides a realistic and challenging environment for evaluating real-time routing performance.

The graph was programmatically generated from OpenStreetMap (OSM), a rich, collaboratively-sourced repository of global geospatial data. The osmnx Python library was utilized to automate the data acquisition and

graph construction process. osmnx downloads the specified map data, projects it, and builds a networkx directed graph, which is crucial for accurately modeling road networks containing one-way streets.

In this graph model:

- * Nodes represent unique OSM identifiers for physical road intersections and endpoints. Each node is attributed with its precise geographic coordinates (latitude and longitude), which are essential for heuristic

- calculations and location-based queries.

- * Edges represent the directed road segments connecting two nodes. Each edge is populated with critical attributes derived from OSM data, most importantly its physical length in meters. This length serves as the

- primary traversal cost, or $g(n)$ value, for the pathfinding algorithm. Additional attributes such as road type (e.g., 'residential', 'primary') and geometry data for curved segments are also retained.

To ensure low-latency startup and consistent performance for the pathfinding service, a critical preprocessing step was implemented. The fully constructed graph is serialized using Python's pickle module and

stored as a .pickle file. This decouples the computationally intensive data acquisition phase from the operational service, allowing the application to load the complete, analysis-ready graph into memory almost instantaneously.

1.2. Core Pathfinding Algorithm

The fundamental pathfinding mechanism employed in this system is the Bidirectional A* algorithm. This approach enhances the efficiency of traditional A* search by simultaneously exploring paths from both the

source and destination nodes, converging at a meeting point in the middle. This bidirectional strategy significantly reduces the search space compared to a unidirectional search, particularly in large graphs like

urban road networks. The heuristic function guiding both forward and backward searches is the Haversine distance. This function calculates the great-circle distance between two geographic points (latitude,

longitude) on a sphere, providing an admissible (never overestimating the actual cost) and consistent estimate of the shortest possible path. The Haversine heuristic provides a pure distance estimate, while the

obstacle avoidance logic is integrated directly into the main search algorithm's decision-making process, as detailed in Section 1.4.

Algorithm: Bidirectional A* Search

Input:

- Graph G
- Source node
- Destination node
- Heuristic function h

Output:

- The shortest path from source to destination, or failure

// Initialization

open_set $\leftarrow$ priority queue

came_from $\leftarrow$ map with keys 'forward' and 'backward'

g_score $\leftarrow$ map with keys 'forward' and 'backward', default value ∞

g_score['forward'][source] $\leftarrow$ 0

g_score['backward'][destination] $\leftarrow$ 0

f_score_fwd $\leftarrow$ h(source, destination)

f_score_bwd $\leftarrow$ h(destination, source)

push (f_score_fwd, source, 'forward') to open_set

push (f_score_bwd, destination, 'backward') to open_set

meeting_node $\leftarrow$ null

best_path_cost $\leftarrow$ ∞

while open_set is not empty do:

- current, direction $\leftarrow$ pop node with lowest f-score from open_set

- opposite_direction $\leftarrow$ 'backward' if direction = 'forward' else 'forward'

```

// Check if a path has been found
if current has been visited from opposite_direction then
    path_cost ← g_score[direction][current] + g_score[opposite_direction][current]
    if path_cost < best_path_cost then
        best_path_cost ← path_cost
        meeting_node ← current

// Early exit condition
if f-score of current > best_path_cost then
    continue // All further paths from this node will be longer

for each neighbor of current in G do:
    // Note: Obstacle avoidance logic is applied here in the final algorithm
    tentative_g_score ← g_score[direction][current] + cost(current, neighbor)
    if tentative_g_score < g_score[direction][neighbor] then
        came_from[direction][neighbor] ← current
        g_score[direction][neighbor] ← tentative_g_score
        target_node ← destination if direction = 'forward' else source
        f_score ← tentative_g_score + h(neighbor, target_node)
        push (f_score, neighbor, direction) to open_set

if meeting_node ≠ null then
    return reconstruct_path(came_from, meeting_node)
else
    return failure

```

1.3. Real-time Obstacle Integration

A key requirement for a practical urban navigation system is the ability to adapt to dynamic changes in network conditions. This system achieves this through a robust, real-time obstacle integration architecture.

The persistence layer for obstacle data is a Supabase cloud database, which allows for real-time updates from various sources. Obstacles are modeled in the database with essential attributes, including the specific graph node_id they affect, a creation timestamp, and their type.

To ensure high performance and minimize I/O latency during pathfinding, the system employs a server-side, time-to-live (TTL) based in-memory cache. This caching layer acts as an intermediary between the

pathfinding algorithm and the database. The obstacle set is fetched from the Supabase database and stored in this local cache. The cache is automatically invalidated and refreshed at a configurable interval

(OBSTACLE_CACHE_TTL, e.g., 300 seconds), ensuring a consistent balance between data freshness and system performance. This architecture guarantees that the pathfinding algorithm operates on a near-real-time

representation of road impediments without incurring the high cost of a database query for every navigation request, thus enabling rapid and adaptive rerouting.

1.4. Obstacle Avoidance Mechanism

This section details the primary enhancement to the Bidirectional A* algorithm: its obstacle-aware avoidance mechanism, which constitutes the core contribution of this research. The strategy employed is *hard

exclusion*, effectively assigning an infinite penalty to any path segment (node or edge) that is deemed impassable or unsafe due to the presence of obstacles. This mechanism ensures that the algorithm strictly

avoids areas affected by real-time impediments.

The implementation of this avoidance strategy is critically dependent on a highly optimized Spatial Index (further elaborated in Section 1.5), which provides rapid, near $O(1)$ lookups for obstacle proximity.

During the pathfinding process, before considering any potential neighbor node or edge for path extension, the algorithm performs three critical checks:

1. Direct Obstacle Check: It verifies if the candidate node itself is explicitly marked as an obstacle (`spatial_index.is_obstacle`).

2. Node Proximity Check: It determines if the candidate node falls within a configurable `obstacle_radius` (e.g., 0.1 km) of any known obstacle (`spatial_index.is_near_obstacle`).

3. Edge Proximity Check: It assesses if the road segment (edge) connecting the current node to the candidate neighbor passes within the specified `obstacle_radius` of any obstacle (`spatial_index.is_edge_near_obstacle`). This check utilizes geometric analysis for precise detection.

If any of these conditions are met, the algorithm immediately discards that neighbor and does not consider it for path extension. This strict avoidance ensures that generated routes are entirely clear of

identified obstacles and their immediate vicinity. The `obstacle_radius` parameter offers flexibility in defining the buffer zone around obstacles, allowing for adaptation to varying obstacle types and safety requirements.

Flowchart Description for Obstacle Avoidance Mechanism

Start: Algorithm considers a `candidate_neighbor` node for path extension from `current_node`.

1. Decision: Is `candidate\_neighbor` a direct obstacle?
 - * (Check: `spatial_index.is_obstacle(candidate_neighbor)`)
 - * IF YES:
 - * Action: Exclude `candidate_neighbor` from further path consideration (infinite penalty).
 - * End Path: Stop processing this `candidate_neighbor`.
 - * IF NO:
 - * Continue to next step.

2. Decision: Is `candidate\_neighbor` within the `obstacle\_radius` of any known obstacle?
 - * (Check: `spatial_index.is_near_obstacle(candidate_neighbor)`)
 - * IF YES:
 - * Action: Exclude `candidate_neighbor` from further path consideration (infinite penalty).
 - * End Path: Stop processing this `candidate_neighbor`.
 - * IF NO:
 - * Continue to next step.

3. Decision: Does the edge from `current\_node` to `candidate\_neighbor` pass within the `obstacle\_radius` of any known obstacle?
 - * (Check: `spatial_index.is_edge_near_obstacle(current_node, candidate_neighbor)`)
 - * IF YES:
 - * Action: Exclude `candidate_neighbor` from further path consideration (infinite penalty).
 - * End Path: Stop processing this `candidate_neighbor`.
 - * IF NO:
 - * Action: Include `candidate_neighbor` in path consideration.
 - * End Path: Continue algorithm with this `candidate_neighbor`.

This section details the primary enhancement to the Bidirectional A* algorithm: its obstacle-aware avoidance mechanism, which constitutes the core contribution of this research. The strategy employed is *hard exclusion*, effectively assigning an infinite penalty to any path segment (node or edge) that is deemed impassable or unsafe due to the presence of obstacles. This mechanism ensures that the algorithm strictly avoids areas affected by real-time impediments.

The implementation of this avoidance strategy is critically dependent on a highly optimized Spatial Index (further elaborated in Section 1.5), which provides rapid, near $O(1)$ lookups for obstacle proximity.

During the pathfinding process, before considering any potential neighbor node or edge for path extension, the algorithm performs three critical checks. This sequential exclusion process is illustrated in Figure

- 1.
- *1.5. Performance Optimizations*

To ensure the real-time responsiveness required for dynamic urban navigation, several performance optimization techniques have been integrated into the system, complementing the enhanced pathfinding algorithm.

These optimizations are crucial for handling the computational demands of large-scale graph traversal and dynamic obstacle integration.

Firstly, a custom Spatial Index (`SpatialIndex` class) was developed to accelerate geographic queries and graph traversal operations, replacing computationally expensive linear searches with efficient spatial

lookups. This index leverages a k-d tree (`scipy.spatial.cKDTree`), a binary space-partitioning data structure, to store the geographic coordinates of all graph nodes. The k-d tree enables highly efficient

operations with average-case complexities of $O(\log N)$ or $O(1)$, including:

- * Node Existence and Retrieval: Rapidly checking if a node exists and retrieving its coordinates.

- * Neighbor Identification: Quickly identifying adjacent nodes.

- * Edge Data Access: Providing $O(1)$ access to edge attributes.

- * Obstacle Proximity Checks: Crucially, it underpins the `is_obstacle`, `is_near_obstacle`, and `is_edge_near_obstacle` functions by allowing fast spatial queries to determine if a node or edge is within the

configurable `obstacle_radius` of any known obstacle. The `is_edge_near_obstacle` function further utilizes geometric primitives (e.g., `shapely.geometry.LineString`, `Point`) for precise spatial intersection

analysis. This significantly reduces the computational overhead during the Bidirectional A* algorithm's graph exploration.

Secondly, a Path Caching mechanism is implemented to avoid redundant computations for frequently requested paths. Once a path is successfully computed by the Bidirectional A* algorithm, it is stored in an

in-memory cache (managed by `cache_utils.py`). The cache key for each stored path is intelligently generated as a hash of the source node ID, destination node ID, and a sorted, immutable representation of the

current set of active obstacles. This ensures that:

- * Identical pathfinding requests (same source, destination, and obstacle configuration) can retrieve results instantly.

- * The cache is automatically invalidated for a given route if the obstacle configuration changes, guaranteeing data freshness.

This strategy drastically reduces computation time for repeated queries, improving overall system throughput and responsiveness, especially in scenarios where users might frequently query paths between the same

points or where the obstacle landscape changes incrementally.

Results